Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ **ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ**

КАФЕДРА **ИНФОРМАЦИОННЫЕ СИСТЕМЫ И ТЕЛЕКОММУНИКАЦИИ (ИУ3)**

НАПРАВЛЕНИЕ ПОДГОТОВКИ **09.03.03 ПРИКЛАДНАЯ ИНФОРМАТИКА**

ОТЧЕТ

по лабораторной работе № 2

Название: Обработка пропусков в данных, кодирование категориальных признаков, масштабирование данных

Дисциплина: Технологии машинного обучения

Студент	ИУ3С-51Б		М.Ю. Гордон
	(Группа)	(Подпись, дата)	(И.О. Фамилия)
Преподаватель			О.Г. Саргсян
		(Подпись, дата)	(И.О. Фамилия)

Москва, 2026

1. Описание задания

Цель лабораторной работы — изучить методы предварительной обработки данных для их дальнейшего использования при построении моделей.

- Выбрать набор данных (датасет), содержащий категориальные признаки и пропуски в данных. Для выполнения следующих пунктов можно использовать несколько различных наборов данных (один для обработки пропусков, другой для категориальных признаков и т.д.)
- Для выбранного датасета (датасетов) на основе материалов лекции решить следующие задачи:
    - обработку пропусков в данных;
    - кодирование категориальных признаков;
    - масштабирование данных.

2. Текст программы

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler

# Загрузка данных
df = pd.read_csv("train.csv")

# Сохраняем количество пропусков до обработки
missing_before = df.isnull().sum()

print("Первые 5 строк:")
print(df.head())

print()
print("Размер датасета:")
print(df.shape)
```

```
print()
print("Типы данных:")
print(df.dtypes)

print()
print("Количество пропусков до обработки:")
print(df.isnull().sum())

# График пропусков до обработки
plt.figure(figsize=(10, 5))

missing_before[missing_before > 0].plot(kind="bar")

plt.title("Количество пропусков до обработки")
plt.xlabel("Признак")
plt.ylabel("Количество пропусков")
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()

# Обработка пропусков

# Заменяем пропуски в Age на медиану
df["Age"] = df["Age"].fillna(df["Age"].median())

# Заменяем пропуски в Embarked на самое частое значение
df["Embarked"] = df["Embarked"].fillna(df["Embarked"].mode()[0])

# Удаляем Cabin из-за большого количества пропусков
df = df.drop(columns=["Cabin"])


print()
print("Пропуски после обработки:")
print(df.isnull().sum())

# Кодирование категориальных признаков
df = pd.get_dummies(
    df,
    columns=["Sex", "Embarked"],
    dtype=int
```

```
)

print()
print("Данные после кодирования категориальных признаков:")
print(df.head())

# Масштабирование данных

numeric_columns = [
    "Pclass",
    "Age",
    "SibSp",
    "Parch",
    "Fare"
]

# Сохраняем данные до масштабирования
before_scaling = df[numeric_columns].copy()

# Создаём StandardScaler
scaler = StandardScaler()

# Выполняем масштабирование
df[numeric_columns] = scaler.fit_transform(
    df[numeric_columns]
)

# Сохраняем данные после масштабирования
after_scaling = df[numeric_columns].copy()


print()
print("Данные после масштабирования:")
print(df[numeric_columns].head())

# График до и после масштабирования

plt.figure(figsize=(12, 6))

plt.subplot(1, 2, 1)

sns.boxplot(data=before_scaling)
```

```
plt.title("До масштабирования")
plt.xticks(rotation=45)


plt.subplot(1, 2, 2)

sns.boxplot(data=after_scaling)

plt.title("После масштабирования")
plt.xticks(rotation=45)


plt.tight_layout()
plt.show()

# Проверка результатов
print()
print("Итоговое количество пропусков:")
print(df.isnull().sum().sum())

print()
print("Средние значения масштабированных признаков:")
print(df[numeric_columns].mean())

print()
print("Стандартные отклонения масштабированных признаков:")
print(df[numeric_columns].std())
```

3. Скриншоты выполнения программы

В результате выполнения программы были получены следующие результаты.

```
Первые 5 строк:
   PassengerId  Survived  Pclass  ...     Fare Cabin  Embarked
0            1         0       3  ...   7.2500   NaN         S
1            2         1       1  ...  71.2833   C85         C
2            3         1       3  ...   7.9250   NaN         S
3            4         1       1  ...  53.1000  C123         S
4            5         0       3  ...   8.0500   NaN         S

[5 rows x 12 columns]

Размер датасета:
(891, 12)

Типы данных:
PassengerId      int64
Survived         int64
Pclass           int64
Name               str
Sex                str
Age            float64
SibSp            int64
Parch            int64
Ticket             str
Fare           float64
Cabin              str
Embarked           str
dtype: object
```

Рисунок 1 - Исходные данные

На рисунке 1 представлены исходные данные датасета Titanic, его размер и типы данных. Также показано количество пропущенных значений. Наибольшее количество пропусков содержится в признаке Cabin.

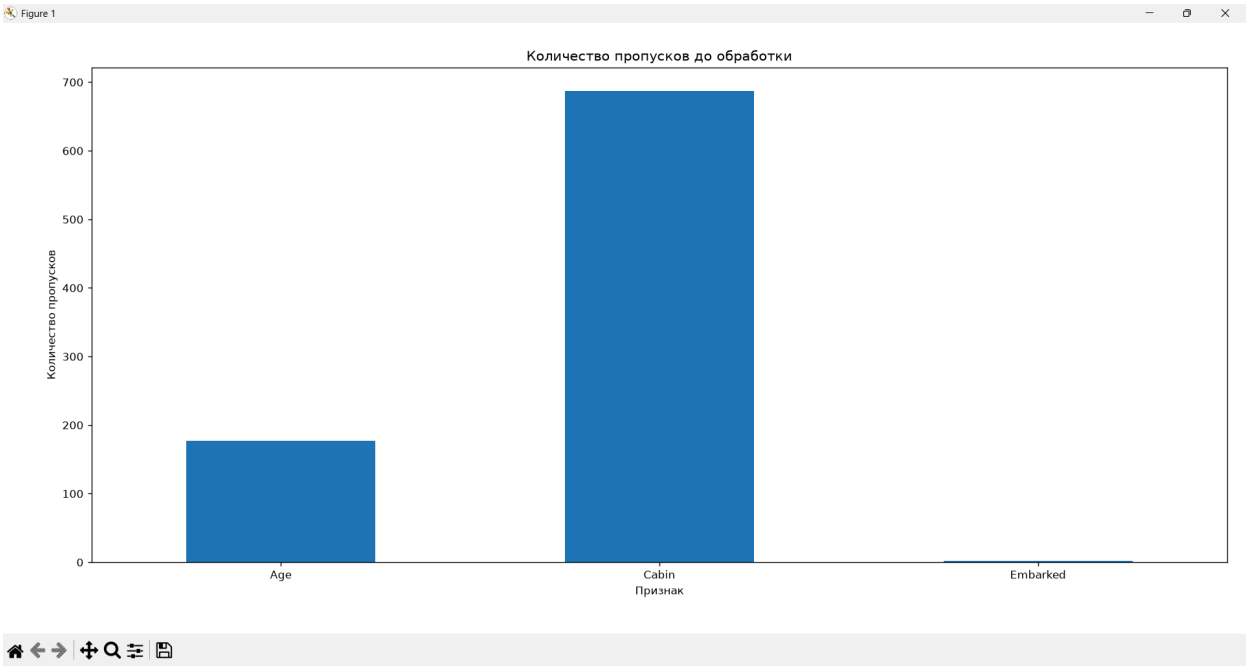


Рисунок 2 - Количество пропущенных значений до обработки

На рисунке 2 показано распределение пропущенных значений до обработки. Видно, что пропуски присутствуют в признаках Age, Cabin и

Embarked. Наибольшее количество пропусков приходится на Cabin.

```
Пропуски после обработки:
PassengerId    0
Survived       0
Pclass         0
Name           0
Sex            0
Age            0
SibSp          0
Parch          0
Ticket         0
Fare           0
Embarked       0
dtype: int64
```

Рисунок 3 - Количество пропущенных значений после обработки

На рисунке 3 показан результат обработки пропущенных значений. Пропуски в признаке Age были заменены медианным значением, а пропуски в Embarked — наиболее часто встречающимся значением. Признак Cabin был удалён из-за большого количества пропусков. В результате в обработанном наборе данных пропущенные значения отсутствуют.

```
Данные после кодирования категориальных признаков:
   PassengerId  Survived  Pclass  ... Embarked_C  Embarked_Q  Embarked_S
0            1         0       3  ...          0           0           1
1            2         1       1  ...          1           0           0
2            3         1       3  ...          0           0           1
3            4         1       1  ...          0           0           1
4            5         0       3  ...          0           0           1

[5 rows x 14 columns]
```

Рисунок 4 - Кодирование категориальных признаков

На рисунке 4 показан результат кодирования категориальных признаков. Признаки Sex и Embarked были преобразованы в числовые столбцы с помощью One-Hot Encoding.

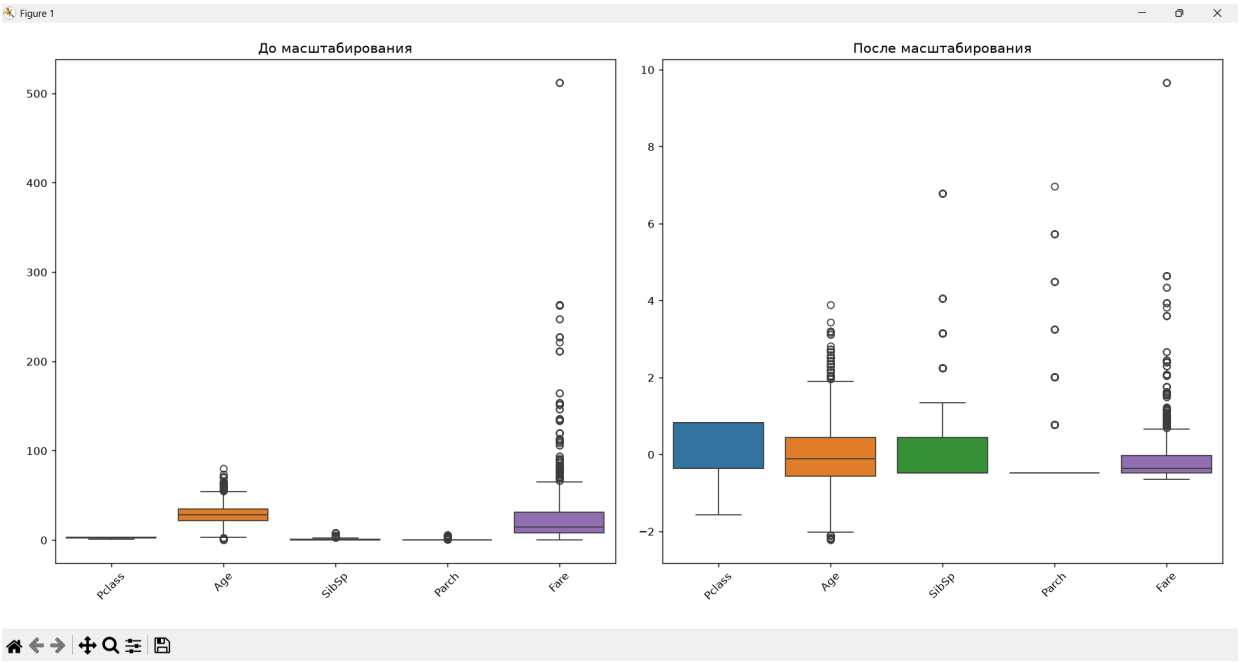


Рисунок 5 - Масштабирование

На рисунке 5 представлено сравнение числовых признаков до и после масштабирования. На графике слева показаны исходные значения признаков Pclass, Age, SibSp, Parch и Fare, которые имеют разные диапазоны и масштабы. Например, значения Fare значительно отличаются по диапазону от значений SibSp и Parch.

На графике справа показаны те же признаки после применения метода StandardScaler. В результате признаки были приведены к сопоставимому масштабу: их средние значения стали близки к нулю, а стандартное отклонение — к единице. Такое преобразование позволяет избежать ситуации, когда признаки с большими числовыми значениями оказывают чрезмерное влияние на дальнейшую обработку данных и построение моделей.

4. Вывод

В ходе лабораторной работы была выполнена предварительная обработка данных датасета Titanic. Были обнаружены и обработаны пропущенные значения: пропуски в признаке Age заменены медианным

значением, в Embarked — наиболее часто встречающимся значением, а признак Cabin удалён из-за большого количества пропусков.

Категориальные признаки Sex и Embarked были преобразованы в числовой формат с помощью One-Hot Encoding. После этого числовые признаки Pclass, Age, SibSp, Parch и Fare были масштабированы с помощью StandardScaler.

В результате был получен подготовленный набор данных без пропущенных значений, с числовыми категориальными признаками и признаками, приведенными к сопоставимому масштабу. Таким образом, были изучены основные методы предварительной обработки данных, которые могут применяться перед построением моделей машинного обучения.